

Ubuntu Backend Engineer & DevOps Roadmap

Learning Roadmap

1. **Start with Linux Fundamentals:** Learn what Linux is (an open-source operating system) and why Ubuntu is popular. Install Ubuntu 24.04 LTS by downloading the ISO, writing it to a USB (using tools like Rufus or Etcher), and booting your PC from it ¹ ². Practice navigating the Ubuntu desktop and opening the **Terminal**.
2. **Learn the File System and Shell:** Explore Ubuntu's filesystem hierarchy (root `/`, `/home`, `/etc`, `/var`, `/usr`, etc.) to understand where files live ³ ⁴. Master basic shell commands (`ls`, `cd`, `cp`, `mv`, `rm`, `mkdir`, etc.) to list, move, and manipulate files ⁵. Try examples like `ls -l`, `cd ~/Documents`, and `mkdir new_folder`.
3. **Manage Files and Permissions:** Learn to edit files from the command line using editors like `nano` or `vim` (`nano file.txt`). Understand file permissions/ownership: use `chmod` to set permission bits and `chown` to change owner/group ⁶. For example, `chmod 755 script.sh` makes a file executable and `sudo chown alice:developers report.pdf` changes its owner. These are vital for security (who can read/write/execute files).
4. **User and Group Accounts:** Learn to create and manage user accounts. Use `sudo adduser username` to add a new user (follow prompts to set password) and `sudo passwd username` to change passwords ⁷. Delete users with `sudo deluser username`. Manage groups with `sudo addgroup groupname` and add users to groups (`sudo adduser username groupname`) ⁸. This is important for permission management (e.g. giving multiple users access to shared files).
5. **Package Management (APT):** Learn to install and update software with APT. Always run `sudo apt update` to refresh the package index, then `sudo apt upgrade` to apply updates ⁹. Install new packages with `sudo apt install package-name` and remove them with `sudo apt remove package-name` ¹⁰ ⁹. Keeping software up-to-date is critical for security and features.
6. **Process and Service Management:** Study how to view and control running processes. Use `ps aux` or `top` to see current processes and resource usage ¹¹. Kill misbehaving processes with `kill <PID>`. For system services (daemons), learn `systemd` with `systemctl`: e.g. `sudo systemctl start nginx`, `sudo systemctl stop nginx`, and `sudo systemctl restart nginx` ¹² ¹³. Check service status with `systemctl status nginx` ¹⁴. Understanding processes and services is key to managing servers.
7. **Networking Basics:** Learn tools to check and configure networking. Use `ping hostname` to test connectivity, and `ifconfig` or `ip a` to view network interfaces (IP addresses). Use `netstat -tuln` or `ss -tuln` to list listening ports and connections. These commands help diagnose network issues (e.g. `ping 8.8.8.8` checks Internet reachability).
8. **Bash Scripting & Environment:** Begin writing simple shell scripts (`.sh` files with `#!/bin/bash` at top). For example:

```
#!/bin/bash
echo "Hello, Ubuntu world!"
```

Save, `chmod +x`, and run it. Use environment variables like `HOME` or custom ones (`export MYVAR=value`) to make scripts more flexible. Check variables with `echo $PATH` or `env` ¹⁵. Scripting automates repetitive tasks (backups, deployments).

9. **System Logging (journalctl):** Learn systemd's journal for logs. Ubuntu uses **systemd's journald** to collect all system and service logs ¹⁶. Use `journalctl` to view logs. For example, `journalctl -xe` shows recent logs and errors ¹⁷, or `journalctl -u ssh` shows SSH service logs. Reading logs helps debug issues and track system activity.
10. **Scheduled Tasks (Cron):** Use **cron** to schedule tasks. Cron runs jobs automatically at specified times. Edit your crontab with `crontab -e` and add lines like: `0 2 * * * /home/alice/backup.sh` (runs daily at 2:00 AM) ¹⁸ ¹⁹. Learn cron syntax (minute, hour, day, month, weekday) – e.g. `30 3 * * 1` runs every Monday at 3:30 AM ²⁰. Cron is vital for regular maintenance (backups, updates).
11. **SSH & Remote Access:** Master **SSH** for secure remote login. SSH (Secure Shell) lets you connect to Linux servers from another machine ²¹. Use `ssh username@server_ip` in the terminal, enter your password (or use keys) ²². Ensure the SSH server is running on Ubuntu (`sudo systemctl start ssh`) ²³. Once connected, you get a remote shell as if locally. Practice copying files with `scp` or setting up key-based auth. SSH is essential for backend and DevOps work.
12. **System Updates & Upgrades:** Learn to keep Ubuntu itself updated. Use `sudo apt update && sudo apt upgrade` regularly to install security patches and package updates ⁹. For full release upgrades (e.g. 24.04 to 24.10), Ubuntu provides an **Upgrade tool**, but even before that ensure everything is updated. This keeps your system secure and feature-rich.
13. **Service Management (systemctl/service):** Go deeper with `systemctl` (from `systemd`) to manage system services. Practice enabling services at boot (`sudo systemctl enable nginx`) and disabling them. Learn `systemctl status` (e.g. `sudo systemctl status ssh`) to check health ¹⁴. You can also use the older `service` command (e.g. `sudo service ssh start`). Managing services is key for running web servers, databases, and other backend services.
14. **Basic Security (Firewall & sudo):** Secure your system. Ubuntu uses **UFW (Uncomplicated Firewall)** as a simple interface to `iptables` ²⁴. For example, enable SSH in the firewall (`sudo ufw allow ssh`) and then turn on the firewall (`sudo ufw enable`) ²⁵ ²⁶. This blocks unwanted traffic. Also use `sudo` for admin tasks: instead of logging in as root, prefix commands with `sudo` to run them as root ⁷ ²⁷. This way actions are logged and controlled.
15. **File System Types & Quotas:** Understand Linux file systems. Ubuntu defaults to **ext4** on its disks (inherited from Debian) ⁴, but others like XFS or Btrfs can be used. Learn how file systems are mounted (see `/etc/fstab`) and basic types. Also learn **disk quotas** to limit users' disk usage ²⁸. For example, quotas can restrict a user to a certain number of megabytes on `/home`. Quotas prevent one user from filling the disk and disrupting others.
16. **Backup & Restore:** Practice backing up data. Use tools like `tar` and `rsync` to copy files. For instance, archive a directory: `tar -czvf backup.tar.gz /home/alice/data` ²⁹. Or use `rsync -av /home/alice/ /mnt/backup/` to sync files efficiently ³⁰. You can schedule these commands with cron. Also learn about full disk backups (using `dd`) and graphical tools (Ubuntu's **Déjà Dup**). Regular backups protect against data loss.

Each step builds on previous ones, ensuring you first master basic Linux usage before tackling DevOps and networking tasks.

Linux Basics & Installation

- **What:** Linux is a free, open-source operating system kernel used by many distributions (distros). Ubuntu is a popular user-friendly Linux distro ³¹.
- **Why important:** As a backend engineer/DevOps, Linux is essential – most servers run Linux. Using Ubuntu 24, you get a stable environment. Installing Ubuntu 24.04 LTS (the latest long-term release) gives you a reliable platform ².
- **Getting started:** Download the Ubuntu 24.04 ISO and create a bootable USB (using Rufus/Etcher) ¹. Boot your PC from this USB and follow the installer's prompts. You can choose "Erase disk and install Ubuntu" to replace an existing OS, or install alongside Windows ³². During install you'll set your timezone and create your first user. After reboot, log in to Ubuntu. You now have Linux running!

Linux Filesystem Hierarchy

- **What:** The Linux filesystem is a single tree starting at root `/`. Common top-level directories include:
 - `/bin`, `/usr/bin`: essential user commands (e.g. `ls`, `cp`).
 - `/etc`: system configuration files.
 - `/home`: home directories for users (e.g. `/home/alice`).
 - `/var`: variable data (logs, mail, spools).
 - `/usr`: user applications and utilities.
 - `/proc` and `/sys`: special virtual filesystems (system and kernel info).
- **Why important:** Knowing this structure helps you find files and config. For example, logs are often under `/var/log`, and system-wide programs under `/usr/bin`. Ubuntu (like Debian) by default uses the ext4 filesystem on `/` ⁴, so the structure above is how files are organized.
- **Example:** Run `ls /` to see the root contents. Explore: `ls /etc` to see config files, `ls /home` for user folders. Remember, Ubuntu can also use ZFS or LVM (as options during install) for advanced setups ³³.

Basic Shell Commands (`ls`, `cd`, `cp`, `mv`, `rm`, `mkdir`)

The `ls` command lists files in a directory ⁵. For example: - `ls`: List files in current directory.

- `ls -l /home`: List files in `/home` with details (owner, permissions).
- `cd /etc`: Change directory to `/etc`. `cd` by itself returns you to your home folder.
- `mkdir new_folder`: Create a new directory named "new_folder".
- `cp file.txt /tmp/`: Copy "file.txt" to the `/tmp/` directory.
- `mv old.txt new.txt`: Rename or move a file from "old.txt" to "new.txt".
- `rm unwanted.txt`: Delete "unwanted.txt". (Be careful: deleted files are gone!)

These commands let you navigate and manage files from the terminal. For example, after `mkdir test`, running `ls` will show the new directory. Such basic commands are the bread-and-butter of working in Ubuntu's shell ⁵.

File Permissions and Ownership (`chmod`, `chown`)

- **What:** Every file has permissions (read/write/execute) for three sets: owner (user), group, others. Ownership has a user and group.
- **Why important:** Permissions control who can read, write, or execute files – critical for security. For example, a script must be executable to run (`rx` bit).
- **Examples:** Use `ls -l` to view permissions (e.g. `drwxr-xr--`). Change them with:
 - `chmod 755 script.sh` – sets permissions to `rwrx-rx-x` (owner can `rw`; group & others can read/execute) ⁶ .
 - `chmod u+x script.sh` – adds execute permission for owner.
 - `sudo chown alice:admin report.pdf` – changes the owner to user “alice” and group “admin” for file “report.pdf” ⁶ .

Always use `sudo` for files you don't own (e.g. in `/etc` or `/usr`). Setting correct permissions keeps files safe. For instance, private data files often are set to 600 (`rw-----`) so only you can read them.

Users and Groups Management

- **What:** Linux is multi-user. Each person has a user account and can belong to groups.
- **Why important:** Proper user/group setup ensures security and organization. Users manage their own files, and groups allow shared access.
- **Examples:**
 - Add a user: `sudo adduser bob` (follow prompts to set password) ⁷ .
 - Delete a user: `sudo deluser bob` ³⁴ .
 - Change a user's password: `sudo passwd alice` .
 - Create a group: `sudo addgroup developers` .
 - Add user to group: `sudo adduser alice developers` ⁸ .

The Ubuntu installer makes the first user a sudoer. You'll often use `sudo` to run admin commands. Managing users and groups lets you control access: for example, adding a user to the `www-data` group might allow them to edit web server files.

Package Management (`apt`)

- **What:** Ubuntu uses the APT package system. `apt` is the command-line tool to install, remove, and update software packages (Debian `.deb` packages) ¹⁰ .
- **Why important:** Installing software and keeping it updated is daily work. You'll install tools, libraries, and server packages with `apt`.
- **Examples:**
 - Update package lists: `sudo apt update` ¹⁰ .
 - Upgrade all packages: `sudo apt upgrade` ⁹ .
 - Install a package: `sudo apt install nginx` (to install the Nginx web server) ¹⁰ .
 - Remove a package: `sudo apt remove nginx` .
 - Search for packages: `apt search keyword` .

For example, running `sudo apt update && sudo apt install git` ensures your index is current and installs Git. APT simplifies software management on Ubuntu.

Process Management (ps, top, kill)

- **What:** Linux processes are running programs. `ps`, `top`, and `kill` help you see and control them.
- **Why important:** You need to monitor resource usage and stop runaway processes.
- **Examples:**
 - `ps aux`: Lists all running processes with details (CPU, memory) ¹¹.
 - `top` or `htop`: Interactive real-time view of processes (sort by CPU/Memory).
 - `kill 1234`: Sends a termination signal to process with PID 1234.
 - `killall httpd`: Kills all processes named "httpd".
 - `sudo service apache2 stop` or `sudo systemctl stop apache2`: Stops a service like Apache.

For instance, if a process hangs, find its PID with `ps aux | grep processname` and kill it. Learning to manage processes ensures system stability.

File Editing (vim, nano, cat)

- **What:** Text editors in the terminal let you create or modify files without a GUI. Common editors are `nano` (easy) and `vim` (powerful). `cat` displays file contents.
- **Why important:** You'll often edit configs or scripts in the shell. Knowing a basic editor is essential.
- **Examples:**
 - `nano notes.txt`: Opens "notes.txt" for editing in Nano (use Ctrl+O to save, Ctrl+X to exit).
 - `vim notes.txt`: Opens in Vim (press `i` to insert text, `Esc`, then `:wq` to save and quit).
 - `cat file.txt`: Prints the file's contents to the terminal. Useful to quickly view small files ³⁵.

For example, to edit the Nginx config, you might run `sudo nano /etc/nginx/nginx.conf`, make changes, and save. Even simple tasks like adding an alias (`alias ll='ls -la'`) are done in an editor.

Networking Basics (ping, ifconfig/ip, netstat)

- **What:** Networking tools help verify connections and view network interfaces.
- **Why important:** As a backend developer/DevOps, you often deal with servers on a network. Checking connectivity and open ports is crucial.
- **Examples:**
 - `ping 8.8.8.8`: Tests connectivity to Google's DNS (checks if the network is up).
 - `ifconfig` (or modern `ip addr`): Shows your machine's IP addresses and interface status.
 - `netstat -tuln` or `ss -tuln`: Lists listening TCP/UDP ports on the server.
 - `curl http://example.com`: Fetches a web page (testing HTTP).

For instance, if a web service is unreachable, `ping` the server IP. Use `ifconfig` to check your server's IP. Use `netstat -tuln` to see if your service's port (e.g. 80) is listening.

Disk Management (df, du, mount, umount)

- **What:** Commands for checking disk usage and mounting drives.

- **Why important:** Servers need enough disk space and proper mounts (for USB drives, partitions, etc.).
- **Examples:**
 - `df -h` : Show disk usage of all mounted filesystems in human-readable form.
 - `du -sh /var/log` : Show the size of `/var/log` directory.
 - `sudo mount /dev/sdb1 /mnt/usb` : Mount the device `/dev/sdb1` at `/mnt/usb`.
 - `sudo umount /mnt/usb` : Unmount it safely.
 - `lsblk` : List block devices and partitions.

For example, run `df -h` before installing software to ensure there's space. If you plug in an external drive, use `lsblk` to find it, then `mount` to access it.

Bash Scripting Basics

- **What:** Bash is the default shell on Ubuntu. You can write scripts (text files with commands) and execute them to automate tasks.
- **Why important:** Automating repetitive tasks (backups, deployments, monitoring) saves time and reduces errors.
- **Examples:** Create a file `script.sh` :

```
#!/bin/bash
echo "Starting backup..."
rsync -av /home/alice/data/ /mnt/backup/
echo "Backup done."
```

Make it executable: `chmod +x script.sh` and run `./script.sh`. This script uses Bash commands to sync a folder. Scripts can use variables (`$HOME`), loops, and conditional logic. Start simple (echoing text, running commands), then learn constructs (for-loops, if-statements).

Environment Variables

- **What:** Environment variables are named values that affect processes. Examples include `PATH`, `HOME`, and custom vars you create.
- **Why important:** They customize the shell and programs (e.g. `PATH` determines where commands are found). You'll often set variables for scripts or tools.
- **Examples:**
 - View all vars with `env` or `printenv` ¹⁵.
 - `echo $HOME` prints your home directory.
 - Set a variable: `export MY_VAR="Hello"`, then `$MY_VAR` refers to it.
 - Add to `PATH`: `export PATH="$PATH:/usr/local/bin"`.

For instance, `PATH` tells the shell where to look for commands. If you install a program in `/opt/tools`, you might `export PATH="$PATH:/opt/tools/bin"` so you can run it by name.

System Logs and `journalctl`

- **What:** Ubuntu (systemd) collects all system and service logs in a centralized binary journal. The `journalctl` command queries this journal ¹⁶.
- **Why important:** Logs are essential for troubleshooting (they show errors, warnings, and info messages). Unlike older `/var/log`, `journalctl` collects everything.
- **Examples:**
 - `journalctl -xe`: Shows the latest logs with emphasis on errors ¹⁷.
 - `journalctl -u nginx`: View logs for the Nginx service.
 - `journalctl --since "2025-08-01" --until now`: Filter by date.

For example, if your web app crashes, run `journalctl -u your-service` to see error messages. Logs will tell you what went wrong (e.g. permission denied, module load failure).

Cron Jobs and Scheduling Tasks

- **What:** Cron is a time-based job scheduler. You write *cron jobs* in a file called a crontab.
- **Why important:** Automatically running tasks (like daily backups) is key in DevOps.
- **Examples:** Edit your user's crontab with `crontab -e`. Each line has 5 timing fields then a command. For example:

```
0 3 * * * /home/alice/backup.sh
```

This runs `backup.sh` every day at 3:00 AM ¹⁸ ¹⁹. Cron fields are **minute hour day month weekday**. Use wildcards (`*`) for "every". Examples:

- `30 2 * * 1 tar -czf /home/alice/week_backup.tar.gz /home/alice` (every Monday at 02:30).
- `0 * * * * echo "Heartbeat"` (hourly). Learn the format and always double-check your syntax. Cron logs (via `journalctl`) confirm tasks ran.

SSH and Remote Access

- **What:** SSH (Secure Shell) allows encrypted remote login to another machine over the network ²¹. Ubuntu has an SSH server (`sshd`).
- **Why important:** As a backend/devops engineer, you will often manage servers remotely. SSH is the standard tool for secure remote access.
- **Examples:**
 - From your local machine, run `ssh alice@server_ip` to connect as user "alice" to the server at `server_ip` ²². You'll be prompted for Alice's password (unless you set up SSH keys).
 - Once connected, you see the Ubuntu welcome message and a shell prompt on the server. Run commands as if you were physically there.
 - Ensure SSH service is running on Ubuntu: `sudo systemctl start ssh` ²³ (it usually is by default). Check `sudo systemctl status ssh`.
 - Use `exit` to log out.

You can also copy files with `scp user@server:/path/to/file .` or mount remote directories with `sshfs`. SSH keys (using `ssh-keygen` and `ssh-copy-id`) allow password-less login, which is safer for automation.

Software Installation and Updates

- **What:** Beyond APT, software can be installed via `snap`, source builds, or Ubuntu's Software Center. Keeping Ubuntu itself updated is crucial.
- **Why important:** Outdated software can have security holes. Ubuntu regularly releases updates for OS and apps.
- **Examples:**
- **Update OS:** Regularly run `sudo apt update && sudo apt upgrade` to get security patches ⁹. You can also run `sudo apt full-upgrade` to handle major upgrades.
- **Install apps:** Besides `apt`, use `sudo snap install package-name` for Snap packages (e.g. `sudo snap install docker`).
- **GUI tools:** Ubuntu's "Software Updater" or "Software & Updates" settings can check for updates.
- **Security patches:** For critical systems, consider `unattended-upgrades` to auto-install security updates.

Example: To ensure everything's current, run:

```
sudo apt update
sudo apt upgrade
```

You'll see packages being upgraded. Reboot if the kernel was updated.

Managing Services (`systemctl`, `service`)

- **What:** Systemd's `systemctl` (and the older `service`) manage background services. Services include web servers (Apache, Nginx), databases (MySQL), etc.
- **Why important:** Knowing how to start/stop/restart and enable/disable services is essential for server management.
- **Examples:**
- **Start/Stop:** `sudo systemctl start nginx.service` starts Nginx ¹². `sudo systemctl stop nginx.service` stops it. You can usually omit ".service".
- **Restart/Reload:** `sudo systemctl restart mysql` (stop and start). `sudo systemctl reload apache2` (re-read config without full restart) ³⁶.
- **Enable/Disable:** `sudo systemctl enable ssh` makes SSH start on boot; `disable` prevents it.
- **Status:** `sudo systemctl status nginx` shows if it's running, loaded, and recent log entries ¹⁴.
- **Legacy:** `sudo service nginx start` is a simpler wrapper (still uses systemd underneath on Ubuntu).

For example, after editing `/etc/nginx/nginx.conf`, you'd run `sudo systemctl reload nginx` to apply changes. If Nginx fails, `systemctl status nginx` will show the error. This command suite is your primary tool for service management ¹² ¹⁴.

Basic Security Practices (Firewall, `sudo`)

- **What:** Basic security involves using firewalls (UFW), limiting root use, and following the principle of least privilege.
- **Why important:** To protect servers from unauthorized access and damage. Even beginners must understand these fundamentals.
- **Examples:**
- **UFW Firewall:** Ubuntu includes **UFW** (Uncomplicated Firewall) to configure iptables easily ²⁴. By default, all incoming is denied and outgoing allowed. For example, allow SSH and HTTP:

```
sudo ufw allow ssh
sudo ufw allow http
sudo ufw enable
```

The first commands allow connections on port 22 (SSH) and 80 (HTTP). Then `sudo ufw enable` turns on the firewall ²⁵ ²⁶. Check with `sudo ufw status verbose`.

- **Sudo:** Avoid logging in as root. Instead, use `sudo` to run privileged commands ⁷. For instance, use `sudo reboot` to reboot (not by logging in as root). Sudo asks for *your* password and logs the action, adding accountability. If needed, you can enable root (`sudo passwd root`), but Ubuntu locks it by default ³⁷.
- **Regular practice:** Keep your account strong, and only run secure scripts/commands as root. A simple rule is: if a command fails with "Permission denied," try it with `sudo` rather than logging in as root.

Disk Quotas and File System Types

- **What:** Disk quotas let you limit how much disk space or how many inodes a user or group can use on a filesystem ²⁸. File system types (ext4, XFS, Btrfs, NTFS, etc.) have different features.
- **Why important:** On multi-user or shared servers, quotas prevent one user from filling the disk and affecting others. Knowing filesystem types helps when performance or advanced features (like snapshotting) are needed.
- **Examples:**
- To enable quotas on ext4, you edit `/etc/fstab` and add `usrquota,grpquota` to the mount options, then use `quotaon -v /` (for root filesystem) ³⁸. After that, `edquota alice` can set Alice's limits.
- Ubuntu (like Debian) uses **ext4 by default** for its root filesystem ⁴. You might encounter XFS (common in enterprise) or others. For example, AWS or cloud images sometimes use `ext4`.
- The command `repquota -a` (after setup) reports current quota usage.

Quotas and choosing the right FS (ext4 for general use, XFS for huge files, etc.) are advanced topics but good to know as you grow.

Backup and Restore Methods

- **What:** Backups safeguard data. Common methods include copying files, taking snapshots, or using backup tools.

- **Why important:** Always back up important data and configurations. In DevOps, you may need to restore from backups after failures.
- **Examples:**
 - **rsync**: Efficiently sync or copy files. Example: `rsync -avz /home/alice/data/ /mnt/backup/` will copy all files from `data` to a backup location ³⁰. The `-a` archive mode preserves attributes; `-v` is verbose; `-z` compresses during transfer.
 - **tar**: Create archives. Example: `tar -czf mybackup.tar.gz /etc /home/alice` creates a compressed archive of `/etc` and `/home/alice` ²⁹. You can then extract it with `tar -xzf mybackup.tar.gz`.
 - **DD**: For full disk cloning: `sudo dd if=/dev/sda of=/dev/sdb bs=4M` copies disk sda to sdb. Use with caution.
 - **Automating Backups**: Put these commands in a script or cron job. For instance, a nightly cron could run an `rsync` to a backup server.
 - **Restore**: To restore from `tar`, use `tar -xzf`. For `rsync`, reverse source and destination.
 - **Tools**: Ubuntu also has tools like **Deja Dup** (a GUI backup) or **duplicit**y for incremental encrypted backups.

Always test your backups by actually restoring a few files to confirm they work. A good rule: “Backups are only good if you can restore from them!”

Each section above builds your skill set step-by-step, covering everything from installing Ubuntu to automating tasks and ensuring system security.

Sources: Authoritative Ubuntu and Linux guides and references have been used throughout, including official Ubuntu docs and DigitalOcean tutorials ³ ¹⁰ ¹⁶ ¹⁸, among others, to ensure accurate and up-to-date information.

¹ ³¹ What is Linux Operating System - GeeksforGeeks

<https://www.geeksforgeeks.org/linux-unix/introduction-to-linux-operating-system/>

² ³² ³³ Install Ubuntu Desktop | Ubuntu

<https://ubuntu.com/tutorials/install-ubuntu-desktop>

³ LinuxFilesystemTreeOverview - Community Help Wiki

<https://help.ubuntu.com/community/LinuxFilesystemTreeOverview>

⁴ filesystem - Why does Ubuntu use ext4 by default? - Ask Ubuntu

<https://askubuntu.com/questions/1384943/why-does-ubuntu-use-ext4-by-default>

⁵ ⁶ ¹¹ ¹⁵ ¹⁷ ²⁷ ²⁹ ³⁰ ³⁵ 50+ Essential Linux Commands: A Comprehensive Guide | DigitalOcean

<https://www.digitalocean.com/community/tutorials/linux-commands>

⁷ ⁸ ³⁴ ³⁷ User management - Ubuntu Server documentation

<https://documentation.ubuntu.com/server/how-to/security/user-management/>

⁹ Upgrade Ubuntu Desktop | Ubuntu

<https://ubuntu.com/tutorials/upgrading-ubuntu-desktop>

10 Install and manage packages - Ubuntu Server documentation

<https://documentation.ubuntu.com/server/how-to/software/package-management/>

12 13 14 36 Manage Systemd Services with systemctl on Linux | DigitalOcean

<https://www.digitalocean.com/community/tutorials/how-to-use-systemctl-to-manage-systemd-services-and-units>

16 How To Use journalctl to View and Manipulate systemd Logs on Linux | DigitalOcean

<https://www.digitalocean.com/community/tutorials/how-to-use-journalctl-to-view-and-manipulate-systemd-logs>

18 19 20 How To Use Cron to Automate Tasks on Ubuntu 18.04 | DigitalOcean

<https://www.digitalocean.com/community/tutorials/how-to-use-cron-to-automate-tasks-ubuntu-1804>

21 22 23 How to Use SSH to Connect to a Remote Server (Step-by-Step Guide) | DigitalOcean

<https://www.digitalocean.com/community/tutorials/how-to-use-ssh-to-connect-to-a-remote-server>

24 25 26 How to Set Up a Firewall with UFW on Ubuntu | DigitalOcean

<https://www.digitalocean.com/community/tutorials/how-to-set-up-a-firewall-with-ufw-on-ubuntu>

28 38 How To Set Filesystem Quotas on Ubuntu 20.04 | DigitalOcean

<https://www.digitalocean.com/community/tutorials/how-to-set-filesystem-quotas-on-ubuntu-20-04>